



GPS-IMU Sensor Module and Outdoor Autonomous Navigation Program

Application Notes for Jaguar Series Robots

Copyright Statement

This document or any portion of it may not be copied or duplicated without the expressed written consent of Dr Robot.

All the software, firmware, hardware and product design accompanying with Dr Robot's product are solely owned and copyrighted by Dr Robot. End users are authorized to use for personal research and educational use only. Duplication, distribution, reverse-engineering, or commercial application of the Dr Robot or licensed software and hardware without the expressed written consent of Dr Robot is explicitly forbidden.

www.DrRobot.com

Contact

General: info@DrRobot.com

Technical Support: support@DrRobot.com

Table of Contents

I.	Jaguar GPS-IMU Sensor Module	4
	GPS	4
	9DOF IMU (Gyro, Accelerometer & Digital Compass)	4
II.	Jaguar Outdoor Autonomous Navigation Program	5
	Software Requirements	5
	Hardware Requirements	5
	DCM functions in DrRobotIMUGPSDCM.dll	6
	Initialize the DCM Algorithm	9
	Using the Navigation Program	10

I. Jaguar GPS-IMU Sensor Module

The Jaguar GPS-IMU Sensor Module integrates a 5Hz GPS receiver, 9DOF IMU sensor module (3DOF Gyro, 3DOF Accelerometer & 3DOF digital compass). Together with Jaguar's outdoor autonomous navigation algorithm, this provides an effective and low cost outdoor localization solution for outdoor mobile vehicle. The data interface to the sensor module is serial or TCP/IP (wire or wireless).

GPS

Input power	5V
Update rate	5 Hz
Sensitivity	- 185dBW minimum
Accuracy	Standard GPS service: Position: Velocity: <= 15m 95% typical 0.1knot RMS steady state WAAS service: Position: <= 3m 95% typical
Output Interface	NMEA 0183, default GPRMC/GPGGA/GPGSA/GPVTG Binary Output

9DOF IMU (Gyro, Accelerometer & Digital Compass)

Input power	5V
Gyro Sensors	ITG3205 Triple-Axis digital output gyro sensor
Accelerometers	3 Axis ADXL345 13bit resolution Max +/-16G
Magnetic Compass	3 Axis HMC5883L magnetometer
Output Frequency	50Hz Output all sensors raw AD data, magnetic sensor will be measured every 220ms

II. Jaguar Outdoor Autonomous Navigation Program

This program, provided with source code will demonstrate the navigation ability of Dr Root's Jaguar series robots using the GPS-IMU sensor module. You are encouraged to modify or develop your own algorithm to use the Jaguar GPS-IMU sensors.

Software Requirements

- **Google Earth program** - could be downloaded from <http://earth.google.com/download-earth.html>. Please follow its installation instruction.
- **DrRobotIMUGPSDCM.dll** – packaged with this program, it should be under execution file folder.
- **OutDoorRobotConfig.xml** – configuration file

The last file should be under "c:/drrobotappfile/" folder.

Hardware Requirements

Any of the Dr Robot's outdoor robots (Jaguar series) with the GPS-IMU sensor module could be controlled by this program. Jaguar series robots include Jaguar V4, Jaguar V2 (with arms), Jaguar Lite, Jaguar 4x4 Wheel and Jaguar 4x4 Wheel with Arm.



You should change the <RobotType> tag in OutDoorRobotConfig.xml to the correct Robot type. The program will then load the corresponding configuration file for the specific robot.

Please note that despite this IMU sensor could output the Roll, Pitch and Yaw estimation. The Yaw estimation is based on magnetometer sensor and may be subject to surrounding magnetic interference such as motors nearby. In this Jaguar navigation program, we won't use it and instead, we provide the DCM estimation based on GPS information from the DrRobotIMUGPSDCM.dll.

DCM functions in DrRobotIMUGPSDCM.dll

All functions in DrRobotIMUGPSDCM.dll are described in DrRobotIMUGPSDCM.cs file. Please note now we use the robot heading estimation from the on board controller. You could implement your own algorithm to estimate the heading by IMU raw data and GPS sensor data.

Here is the list:

```
/// </summary>
/// <param name="dir">left motor direction, if left motor go forward, encoder value
is increasing, dir = 1, otherwise, dir = -1</param>
/// <param name="wheelCnt">wheel spin one circle, encoder count number</param>
/// <param name="wheelR">wheel Radius</param>
/// <param name="wheelDis"> distance between left/right wheel</param>
/// <returns></returns>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static extern int SetEncoderParameter(int dir, int wheelCnt, double wheelR, double
wheelDis);

/// <summary>
///this function is for GPS sensor data update
/// </summary>
/// <param name="cog">cog -- course over ground, radian, -pI ~ PI</param>
/// <param name="vog">vog -- velocity over ground, unit: m/s</param>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static extern void UpdateGPS(double cog, double vog);

/// <summary>
/// this function will estimate position based on latest encoder information
/// </summary>
/// <param name="leftEncoder">current left encoder reading</param>
/// <param name="rightEncoder">current right encoder reading</param>
/// <param name="estPos">pointer to current position estimation, estPos[0] -- E,
estPos[1] -- N, estPos[2] -- Orientation(Yaw)</param>
/// <returns></returns>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static unsafe extern int EncoderEstimate(int leftEncoder, int rightEncoder, double*
estPos);

/// <summary>
///this function will use GPS latitude and longitude reading to update estimated
position
/// </summary>
/// <param name="latitude">unit: degree</param>
/// <param name="longitude">unit : degree</param>
/// <param name="estPos">pointer to current position estimation, estPos[0] -- E,
estPos[1] -- N, estPos[2] -- Orientation(Yaw)</param>
/// <returns></returns>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static unsafe extern int GPSUpdatePosition(double latitude, double longitude, double*
estPos);

/// <summary>
/// these 5 functions are used for coordinate transform, this function must be
called first
/// GPS (latitude, longitude, altitude) <---> ECEF (XYZ) <----> Navigation ENU
(E,N,U)
/// we need first set reference latitude, longitude (ENU (0,0,0) point
```

```

///
/// </summary>
/// <param name="latitude"> unit: degree</param>
/// <param name="longitude"> unit: degree</param>
/// <param name="altitude">0</param>
/// <param name="xyzr">reference point in ECEF(XYZ)</param>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static unsafe extern void SetRefXYZ(double latitude, double longitude, double
altitude, double* xyzr);

/// <summary>
/// this function is for LLH to XYZ
/// </summary>
/// <param name="lat">unit: degree</param>
/// <param name="lng">unit: degree</param>
/// <param name="h">0</param>
/// <param name="XYZ">position in ECEF</param>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static unsafe extern void LLH2XYZ(double lat, double lng, double h, double* XYZ);

/// <summary>
/// this function is for XYZ to ENU
/// </summary>
/// <param name="XYZr">reference point XYZ</param>
/// <param name="XYZ">point XYZ(ECEF)</param>
/// <param name="ENU">point in ENU</param>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static unsafe extern void XYZ2ENU(double* XYZr, double* XYZ, double* ENU);

//
//
//
// setENU[2] --U 0 forced as "0" in function
/// <summary>
/// this function is ENU 2 XYZ
/// setENU[0] - E
/// setENU[1] -- N
/// setENU[2] --U 0 forced as "0"
/// </summary>
/// <param name="setENU"></param>
/// <param name="calXYZ"></param>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static unsafe extern void ENU2XYZ(double* setENU, double* calXYZ);

//
//
/// <summary>
/// XYZ to LLH
/// latitude, longitude unit: degree, altitude = 0
/// </summary>
/// <param name="XYZ"></param>
/// <param name="calLLH"></param>
[DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
static unsafe extern void XYZ2LLH(double* XYZ, double* calLLH);

```

```

    /// <summary>
    /// this function will calculate the distance between two points
    /// (latitude1, longitude1) to (latitude0, longitude0)
    /// </summary>
    /// <param name="latitude1"></param>
    /// <param name="longitude1"></param>
    /// <param name="latitude0"></param>
    /// <param name="longitude0"></param>
    /// <returns></returns>
    [DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
    static extern double DistanceLL(double latitude1, double longitude1, double
latitude0, double longitude0);

    /// <summary>
    /// this function will calculate the heading direction from (latitude1, longitude1)
to (latitude2, longitude2)
    /// the return value is in radian, -PI ~ PI, 0 is point to north, PI/2 is point to
East
    /// </summary>
    /// <param name="latitude1"></param>
    /// <param name="longitude1"></param>
    /// <param name="latitude2"></param>
    /// <param name="longitude2"></param>
    /// <returns></returns>
    [DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
    static extern double BearingToLL(double latitude1, double longitude1, double
latitude2, double longitude2);

    /// <summary>
    /// this function will estimate driving direction, based on current estimated
position and preset path
    /// the preset path is defined by (startENU) ---> endENU
    /// the return value is robot driving direction
    /// </summary>
    /// <param name="estPos"></param>
    /// <param name="distanceError"></param>
    /// <param name="startDir"></param>
    /// <param name="heading"></param>
    /// <param name="startENU"></param>
    /// <param name="endENU"></param>
    /// <returns></returns>
    [DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
    static unsafe extern double EstimateDrvDir(double* estPos, double
distanceError, double startDir, double heading, double* startENU, double* endENU);

    /// <summary>
    /// this function will set driving parameters
    ///
    /// </summary>
    /// <param name="targetTH">distance 2 target threshold</param>
    /// <param name="pathTH">distance 2 preset path threshold</param>
    /// <param name="angleTH">unit radian, set the minimum angle for robot
adjustment</param>
    [DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
    static extern void SetDrvParameter(double targetTH, double pathTH, double angleTH);

    //

```

```

    /// <summary>
    /// this function will return distance between current XYZ and preset path(startXYZ-
-endXYZ)
    /// </summary>
    /// <param name="curXYZ"></param>
    /// <param name="startXYZ"></param>
    /// <param name="endXYZ"></param>
    /// <returns></returns>
    [DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
    static unsafe extern double Distance2Path(double* curXYZ, double* startXYZ, double*
endXYZ);

    /// <summary>
    /// this function is setting GPS position estimate parameter
    /// position = encoder_position + kp *(gps_position - encoder_position)
    /// </summary>
    /// <param name="kp">now we set it as 0.5, if ground is very slip, you can trust GPS
more, set as 0.8 or 1</param>
    /// <param name="diffTH"> if abs( distance(gps_position - pre_gps_position)-
distance(encoder_position- pre_encoder_position) ) > diffTH, we did not use GPS
estimation</param>
    [DllImport("DrRobotIMUGPSDCM.dll", SetLastError = true)]
    static extern void SetGPSPositionUpdateParameter(double kp, double diffTH);

```

Initialize the DCM Algorithm

In form load event, you need to initialize the DCM algorithm.

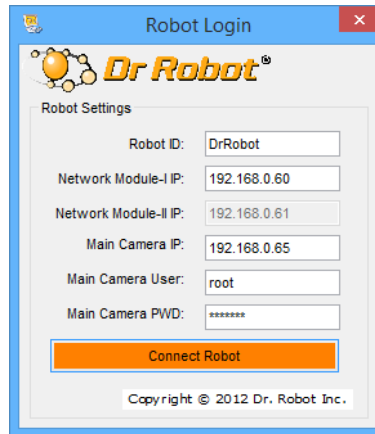
```

    // set Encoder parameter
    SetEncoderParameter(MOTDIR, WHEEL_CNT, WHEEL_R, WHEELDIS);
    // set driving parameter
    SetDrvParameter(dis2TargetTH, dis2PathTH, drvAngleTH);
    // set some parameters
    SetGPSPositionUpdateParameter(0.5, 2.0); //2.0

```

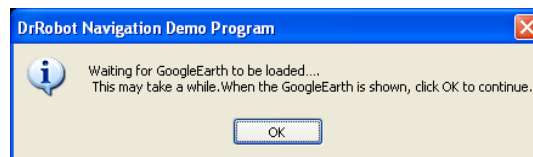
Using the Navigation Program

Login Windows

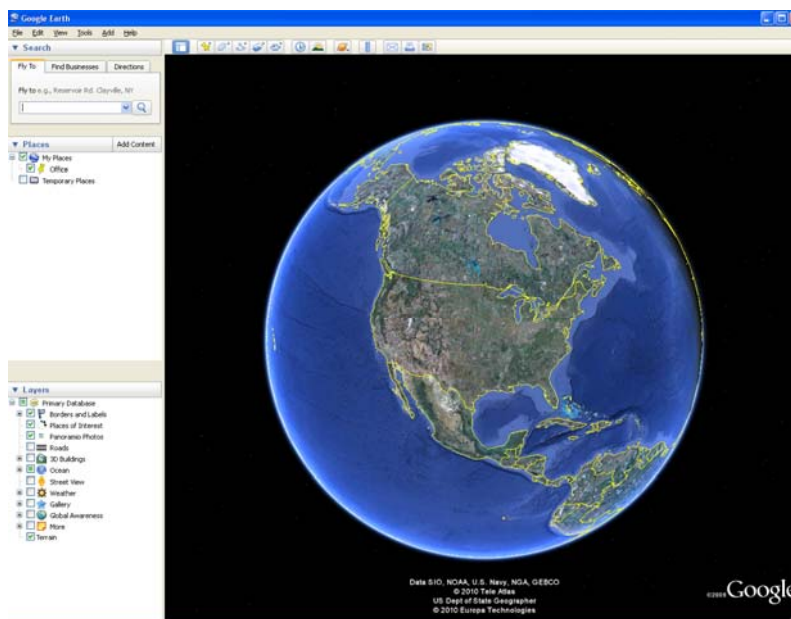


It will read all the configuration information from outdoorrobotconfig.xml (under c:\DrRobotAppFile\).

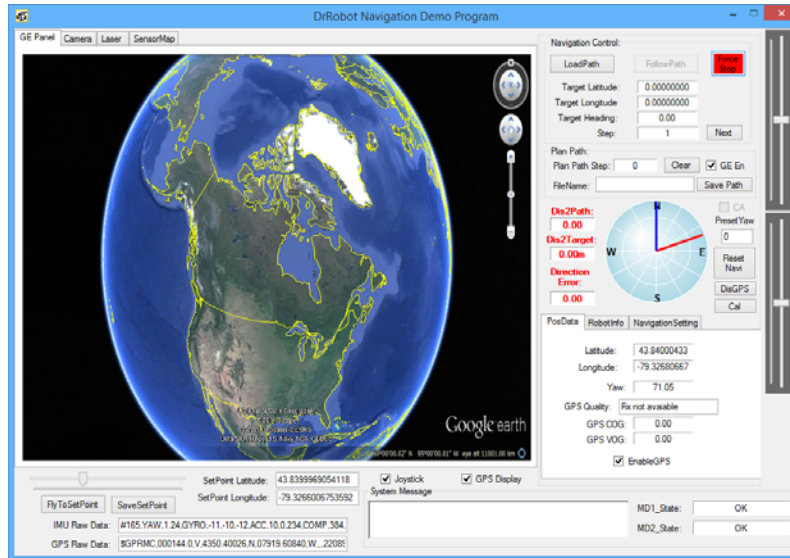
Program will also call up the Google Earth program and wait until the loading finished.



Google Earth supports offline use but you may need to obtain the map online first. If it cannot get access to Internet, it will take some time to re-try connecting with Google Earth website. You have to wait because you cannot get correct Latitude/Longitude position by mouse clicking on map if the map loading is not finished. After you see the Google Earth is loaded as shown below, you should click the "OK" button on previous window.



After the Navigation program is loaded, you should see



FlyToSetPoint

"FlyToSetPoint" button will bring you to the location (latitude/longitude) specified in outdoorrobotconfig.xml. This is the location you would like the map to center and show around. You should modify this location according to your location. This could be done by inputting the value in this xml file or navigating on Google Earth map to your interested point, then clicking "SaveSetPoint" button. The location value of the map center will then be saved to the outdoorrobotconfig.xml when program is closing.

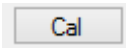
You can use this track bar  to zoom in or out on Google Earth.

IMU/GPS sensor

You will get raw IMU/GPS sensor information in

```
IMU Raw Data: #5,YAW,0.27,GYRO,-30,-29,-26,ACC,19,-13,236,COMP,413,-
GPS Raw Data: $GPRMC,180904.6,A,4350.39871,N,07919.61019,W,000.01
```

Calibrating IMU Gyro sensor

 After receiving IMU/GPS sensor data, you have to keep robot still and click Calibrate  button to calibrate the Gyro sensor offset.

```
MD1_State: +ESTOP
MD2_State: +ESTOP
```

will display 2 motor driver board state. After first start up program, it normal will be in "ESTOP" (Emergency Stop State), you could use Joystick(same as the Dr Robot Jaguar control

program) to resume the motor driver board or click  button.

Wait for GPS getting fixed data.

With ☒ Joystick is checked, you can use Joystick to control robot to move around (same as the Dr Robot Jaguar control program) and get the robot location trace (blue trace on Google Earth) with ☒ GPS Display checked.

⚠ When first start, you may need to allow enough time for the GPS module to obtain correct position. When ready, GPS module gets "GPS fix available" in textbox **GPS Quality:** .

⚠ To make sure robot know where it locates and which way it faces, you need to initialize the Yaw value:

1). GPS COG(course over ground" direction) is the moving direction of antenna, i.e. if robot move backwards, the robot Orientation(Yaw) should be opposite to the COG, however the algorithm does not know this.

2). Therefore, in order to obtain the correct Robot orientation, you should drive robot forward, and when you

observe the correct "COG" value, input it in and click button. This will initialize the robot orientation.

Load Preset Path

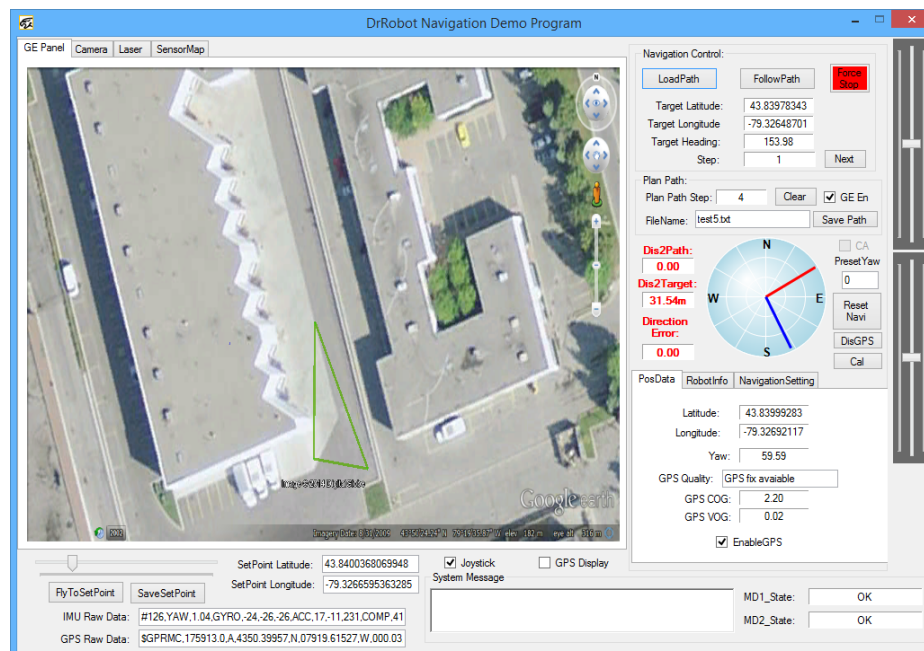
After program knows where the robot locates, you could load a preset path (if you have edited one) by clicking "LoadPath".

Path file format is plain text file with format: "latitude,longitude,O'.

Here is a sample file:

```
43.8549744652762,-79.3600978241786,0
43.8548200333177,-79.3598632398461,0
43.854896224405,-79.3598285570107,0
43.8550948199643,-79.3599150875869,0
43.8549744646843,-79.3600961110703,0
```

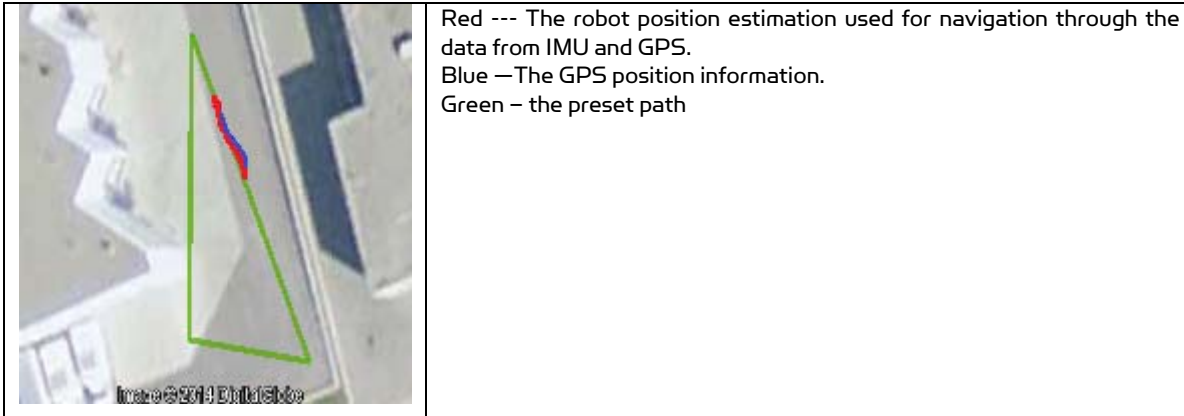
Now the program will show as below:



Navigation

First, you should uncheck the ☒ Joystick box, then you can click "Follow Path" button. The robot will then run along the preset path.

You will see red/blue trace on the Google Earth interface.

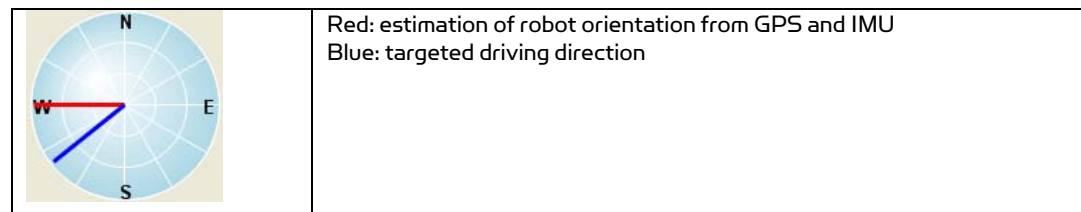


In navigation process, you could click the button to modify the waypoints manually. It is handy when you prefer the robot to skip one of few waypoints for whatever reasons.

You will also find other navigation information, such as:

Dis2Path: 0.58	Dis2Path: distance between current position and plan path
Dis2Target: 14.35m	Dis2Target: distance between current position and target waypoint
Direction Error: 157.41	Direction Error: the angle between the robot orientation and targeted driving direction

Compass display



You will find that the navigation algorithm will adjust driving direction to guide the robot to run along the path. You will also find other robot information, such as:

☒ MP	Temp(°C)	Enc Vel	Enc Pos	Power	Current	Motor temperature
Motor1:	30.8	0	7926	0	0	Encoder readings
Motor2:	31.0	0	-8236	0	0	ACK is the motor driver board acknowledge signal after receive command, "+" is right. "-" is wrong.
Motor3:	29.7	0	7617	0	0	Power: PWM value sent to motor from motor driver board
Motor4:	30.1	0	-7665	0	0	⚠ You may want to stop robot when the motor (s) get very hot (e.g. continuously over 65°) to avoid damaging the motors.
Enc Vel(m/s):	0.00	MD1_ACK:	+			
Voltage(V):	22.9	MD2_ACK:	+			

DCM/Navigation Settings

Dis2TTH: 1.00	DisGPSTH: 18	Dis2TTH: Once reaching within this value, robot will assume current waypoint reached and proceed to next waypoint.
Dis2PathTH: 0.20	DisGPSDis: 2	Dis2PathTh: When position error exceeding this value, robot starts adjusting driving direction to keep robot on the preset path.
DrvAngleTH: 5.00	AdjT: 1	DrvAngleTH: When orientation error exceeding this value, robot starts adjusting driving PWM to keep robot heading to the targeted direction.
MaxDiff: 1000	CtrlT: 4	MaxDiff: The maximum differential PWM value between left/right wheels. This value will make sure robot could turn when needed. Larger value is needed when terrain is rough.
DrvPower: 60.00		AdjT: The value for adjusting turning power
		CtrlT: The control period. The control command is in IMU data reading process. So it is in 80ms control period based on 4*20ms
		DisGPSTH: The delay from GPS will drag the estimation of algorithm behind the actually movement, when robot approaches waypoint and turns to next waypoint, this program could disable use of GPS data for a while. You can set this value to "0" simply to disable this function (default). These value divided by 5 (5Hz GPS) will be actual disable time, e.g. "10" means disabling GPS for 2 seconds.
		DisGPSDis: If distance between current position and target waypoint is less than this value, "disable GPS" function will start unless DisGPSTH=0.

Emergency Stop Clicking  button will stop robot.

Plan your Path

After starting the robot, you should use Joystick control to drive the robot around in open area and wait for the on-board GPS to stabilize and obtain correct position and orientation. This may take one to few minutes.

Then, you need to click "Clear" button to reset plan path step to "0". Plan Path Step:

After unchecking the ☒ GE En checkbox, you could use mouse to click on the desired waypoint (set points) on Google Earth. Note: the first point (the starting point) will not be displayed until you select the second waypoint.



After you finish all waypoints, you will see something like this:



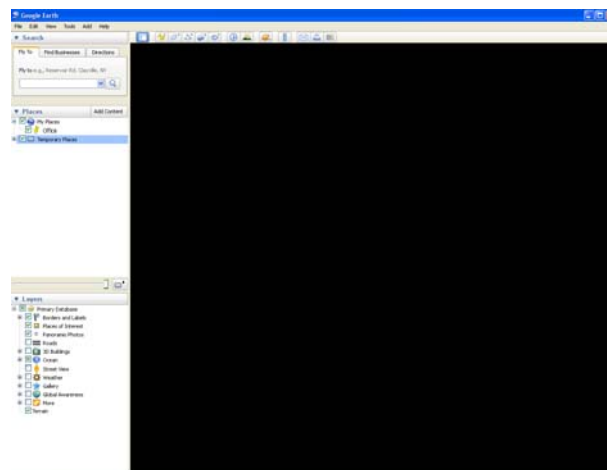
At the end, you could save the path to c:\DrRobotAppFile for later use with the name you prefer.

FileName:

Before loading path file, you should manually delete KML data described in next session to clear up Google Earth window.

Clear KML data

All the path and navigation traces are displayed on Google Earth by KML data. At this moment, Google Earth does not provide method to clear these KML data, there are some risk of memory leak if not being cleared. Therefore, you need to manually clear these KML Data.



On Google Earth program, you could right click on "Temporary Places", and choose "Delete Contents".

Joystick control

These track bars will show Joystick control power for left/right wheel when ☒ Joystick checked.

Battery

You could obtain battery voltage reading from Voltage(V): 23.41 box.

⚠️The robot uses Lion-Polymer battery, so you'd better stop the robot when voltage is below marked voltage (6S = 22.2V) to protect the battery life.

Closing program

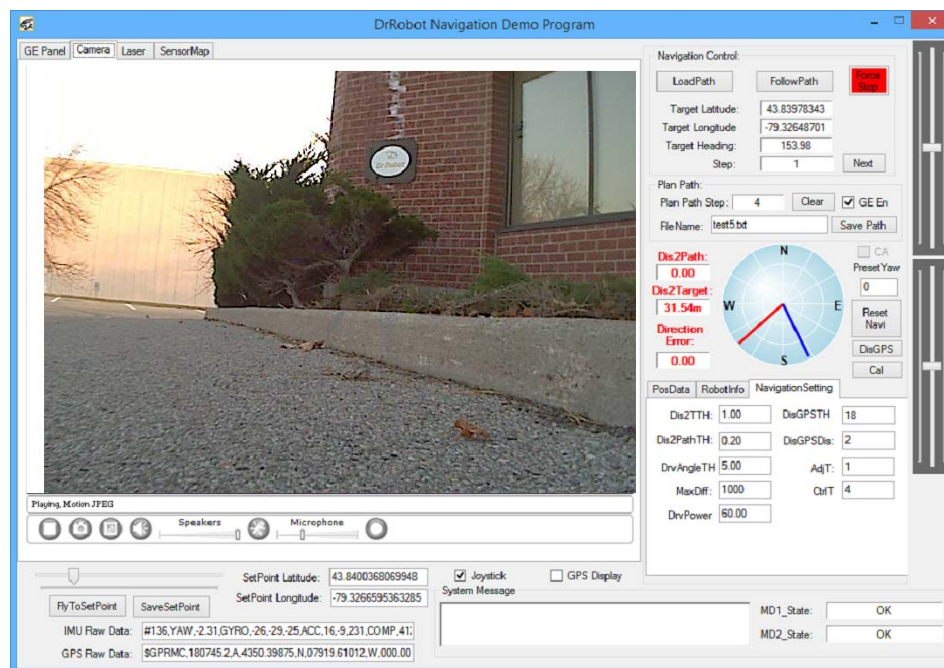
This navigation program will also close Google Earth program on exiting, but we highly recommend that you should double check to make sure Google Earth is actually closed by using "Windows Task Manager", otherwise you cannot get Google Earth Display when you start this navigation program again.

Recording Raw sensor data

You can record raw IMU/GPS/Encoder Reading data by clicking button. The raw data will be saved to "c:\DrRoboAppFile" folder, with file name GPSIMURec*.txt.123

Switch to Camera Image Display

Click on tab to show the currently camera image.



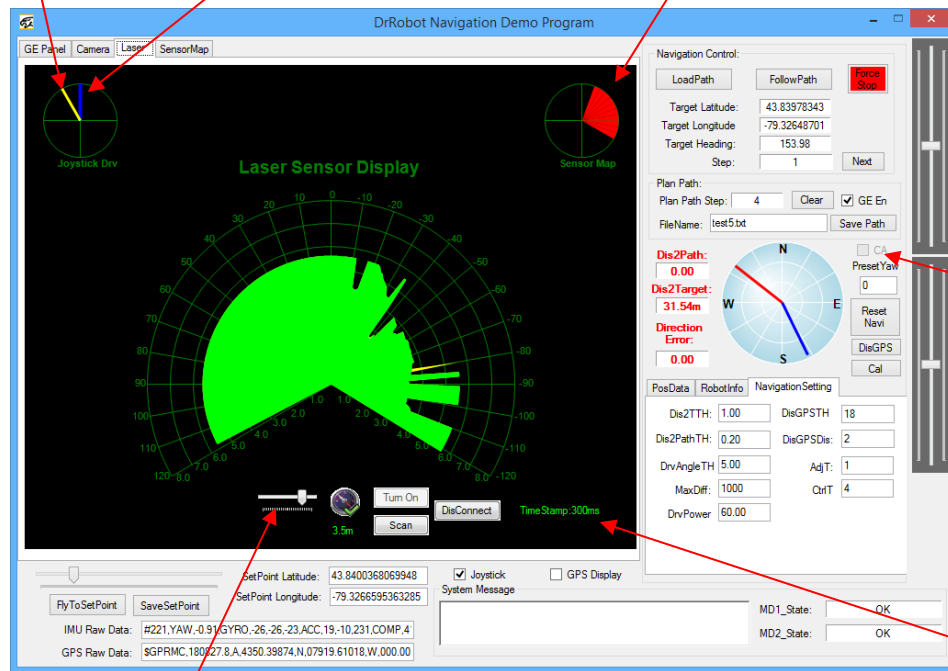
Switch to Laser Scan Display

Click on tab to show the currently laser scan display.

Yellow: Driving Direction(Modified by sensor map)

Blue: Gamepad Driving Direction

Polar Sensor Map



Enable Collision Avoidance

This trackbar will modify the detect object distance

Switch to Sensor Map Display

Click on **GE Panel** **Camera** **Laser** **SensorMap** tab to show the currently sensor map.

